

UFRN - Universidade Federal do Rio Grande do Norte

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 15 - PROGRAMAÇÃO EM MEMÓRIA DISTRIBUÍDA

1. Enunciado

A tarefa consiste em simular a difusão de calor em uma barra unidimensional dividida entre dois ou mais processos MPI, cada um responsável por um trecho contíguo da barra e por duas células fantasma usadas na troca de bordas com os vizinhos. A partir dessa base, foram implementadas e comparadas três estratégias de comunicação: comunicação bloqueante com MPI_Send/MPI_Recv; comunicação não bloqueante com MPI_Isend/MPI_Irecv seguida de MPI_Wait; e uma terceira versão que usa MPI_Isend/MPI_Irecv com MPI_Test para consultar a comunicação periodicamente enquanto os pontos internos são calculados, buscando sobrepor computação e comunicação.

2. Fundamentação

Cada processo armazena um vetor local de $m + 2$ posições: m pontos reais índices 1 a m mais duas células fantasma índices 0 e $m+1$, que recebem os valores de fronteira dos vizinhos esquerdo e direito a cada passo de tempo. A atualização de cada ponto segue o esquema explícito FTCS:

$$u_i^{n+1} = u_i^n + c (u_{i-1}^n - 2u_i^n + u_{i+1}^n)$$

onde $c = 0,25$ é o coeficiente de difusão utilizado. Os pontos estritamente internos ($2 \leq i \leq m-1$) usam apenas valores que já pertencem ao processo e podem ser atualizados a qualquer momento, os pontos de borda locais ($i = 1$ e $i = m$) dependem das células fantasma recebidas dos vizinhos e só podem ser atualizados depois que essa recepção é concluída. Nas duas extremidades globais da barra, o vizinho correspondente é MPI_PROC_NULL e a célula fantasma permanece em zero indefinidamente, funcionando como uma condição de contorno de Dirichlet nula fora do domínio físico. A condição inicial usada foi um pulso de calor de amplitude 100 no terço central do domínio global, com 0 no restante; a barra de N pontos é distribuída entre os P processos da forma mais uniforme possível, repartindo o resto da divisão entre os primeiros processos para não perder pontos quando N não é múltiplo de P .

2.1 Troca bloqueante (MPI_Send/MPI_Recv)

A implementação bloqueante troca a borda direita e depois a esquerda com os vizinhos, mas diferentemente de uma ordem fixa de envio/recebimento igual para todos os processos, que pode causar deadlock em uma cadeia 1D dependendo do protocolo (eager ou rendezvous) e do tamanho da mensagem a ordem das chamadas MPI_Send/MPI_Recv foi escalonada pela paridade do rank: processos de rank par enviam primeiro e recebem depois, processos de rank ímpar recebem primeiro e enviam depois. Isso garante que, em qualquer ponto da cadeia, sempre exista um processo pronto para receber enquanto o vizinho envia, eliminando o risco de deadlock independentemente do tamanho da mensagem.

2.2 Comunicação não bloqueante com MPI_Wait

Nesta versão, as quatro operações não bloqueantes dois envios com MPI_Isend e dois recebimentos com MPI_Irecv são iniciadas de uma vez, e o processo aguarda imediatamente a conclusão de cada uma com MPI_Wait antes de calcular qualquer ponto. Não há sobreposição do ponto de vista de tempo de CPU, o processo fica ocioso durante toda a comunicação, de forma equivalente à versão bloqueante.

2.3 Sobreposição com MPI_Isend/MPI_Irecv + MPI_Test

A terceira versão também inicia as quatro operações não bloqueantes, mas em vez de esperar, calcula os pontos internos $[2, m-1]$ em blocos de 2000 pontos, chamando MPI_Testall após cada bloco para verificar, sem bloquear, se a comunicação das bordas já terminou. Ao final do laço de blocos, se a comunicação ainda não tiver sido confirmada como concluída, um MPI_Waitall garante essa conclusão antes de calcular os dois pontos de borda $i = 1$ e $i = m$, que dependem das células fantasma recém-recebidas.

2.4 Ordem das operações em cada passo

Versão	Passo 1	Passo 2	Passo 3
Send/Recv	Enviar/receber borda direita, depois esquerda (ordem por paridade do rank)	Calcular todos os pontos 1..m	Trocar ponteiros
Isend/Irecv + Wait	Iniciar 4 operações (2 envios, 2 recebimentos)	Wait nas 4 operações, depois calcular todos os pontos 1..m	Trocar ponteiros
Isend/Irecv + Test	Iniciar 4 operações	Calcular interior $[2..m-1]$ em blocos de 2000, com Testall a cada bloco; Waitall se ainda pendente; calcular bordas 1 e m	Trocar ponteiros

3. Experimento

O job 2118946 foi executado na partição amd-512 do NPAD, com dois processos em nós distintos um processo por nó. O programa foi compilado apenas com "mpicc -o barra barra1d.c", sem flags de

otimização adicionais. A barra tinha 1000000 de células, distribuídas igualmente entre os dois processos 500000 células locais cada, mais duas fantasmas. A condição inicial foi o pulso central descrito na Seção 2, com $c = 0,25$. Foram executados 1000 passos de tempo para cada uma das três versões, nessa ordem, dentro do mesmo job e com a mesma configuração de entrada. O tempo medido com MPI_Wtime abrange apenas o laço dos 1000 passos e a barreira final, não incluindo alocação, inicialização da barra nem a redução da soma global, que foi impressa apenas como verificação de consistência entre versões.

4. Resultados

Versão	Tempo (s)	Soma global
1 — Send/Recv (bloqueante)	1,884325	33 333 400,000000
2 — Isend/Irecv + Wait	1,900909	33 333 400,000000
3 — Isend/Irecv + Test (overlap)	1,821498	33 333 400,000000

Tabela 1: Tempos do job 2118946 (uma execução por versão, mesma condição inicial, mesmo job).

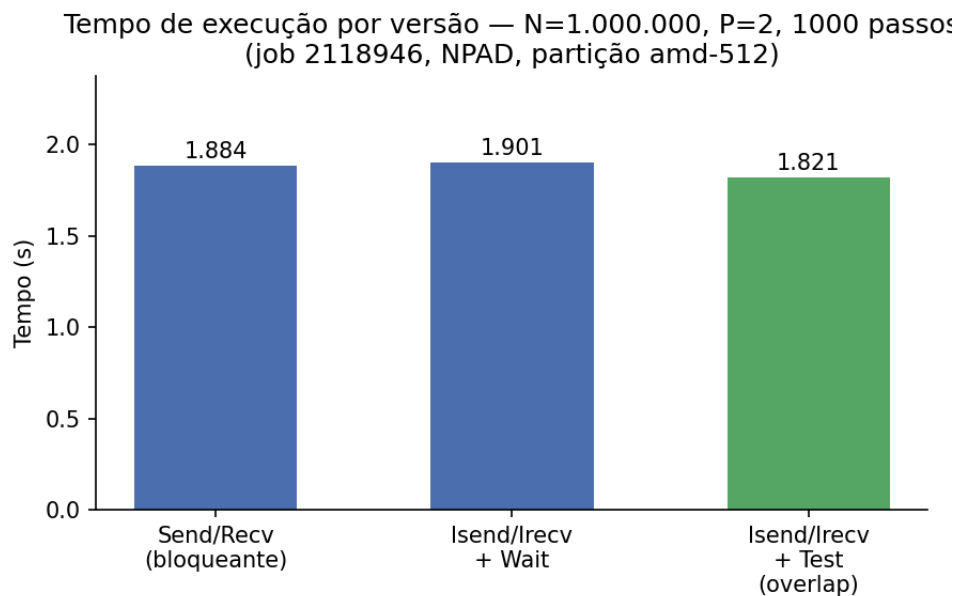


Figura 1: Tempo de execução por versão (job 2118946).

A soma global impressa ao final foi 33 333 400,000000 nas três versões, confirmando que a troca de bordas preserva o mesmo resultado físico independentemente da estratégia de comunicação usada.

Como referência adicional de variabilidade entre submissões, uma execução isolada anterior da versão 3 job 2118944, mesma configuração mediu 1,677861 s cerca de 8% mais rápida que a mesma versão medida junto às outras duas no job 2118946 1,821498 s. Essa diferença entre execuções do mesmo código, na mesma partição, é esperada em um cluster compartilhado como o NPAD, e reforça a limitação discutida a seguir: os valores desta tabela vêm de uma única execução por versão, não de uma mediana de múltiplas repetições.

5. Resultados do experimento de escalonamento

O experimento com o script do Anexo C compilou o mesmo programa com -O3 -march=native e executou 1000 passos sobre N=1 000 000 células para P = 2, 4, 8, 16 e 32. Foram realizadas três execuções consecutivas de cada modo para cada valor de P, totalizando 45 medições. O script solicita

dois nós na partição amd-512 e até 32 tarefas. A Tabela 2 apresenta os tempos individuais e a mediana de cada combinação; a ordem das execuções foi fixa, agrupada por P e por modo.

P	Modo	Rep. 1 (s)	Rep. 2 (s)	Rep. 3 (s)	Mediana (s)
2	1 — Send/Recv	0,544830	0,567140	0,562369	0,562369
2	2 — Isend/Irecv + Wait	0,692808	0,685420	0,736029	0,692808
2	3 — Isend/Irecv + Test	0,573124	0,600866	0,561517	0,573124
4	1 — Send/Recv	0,310695	0,309094	0,359303	0,310695
4	2 — Isend/Irecv + Wait	0,365572	0,356090	0,356481	0,356481
4	3 — Isend/Irecv + Test	0,275765	0,278092	0,260292	0,275765
8	1 — Send/Recv	0,156028	0,149582	0,134940	0,149582
8	2 — Isend/Irecv + Wait	0,172517	0,168125	0,170687	0,170687
8	3 — Isend/Irecv + Test	0,137490	0,129583	0,137842	0,137490
16	1 — Send/Recv	0,083914	0,078827	0,085555	0,083914
16	2 — Isend/Irecv + Wait	0,092220	0,101066	0,094285	0,094285
16	3 — Isend/Irecv + Test	0,111260	0,082841	0,069189	0,082841
32	1 — Send/Recv	0,075925	0,054101	0,071243	0,071243
32	2 — Isend/Irecv + Wait	0,070837	0,085743	0,105726	0,085743
32	3 — Isend/Irecv + Test	0,067477	0,051405	0,070558	0,067477

Tabela 2: Tempos das três repetições e medianas por número de processos e modo (s). Todas as 45 execuções retornaram soma global de 33 333 400,000000.

A mediana do modo 3 foi menor que a do modo 1 em P=4 (11,2%), P=8 (8,1%), P=16 (1,3%) e P=32 (5,3%); em P=2, o modo 1 foi 1,9% mais rápido que o modo 3. O modo 2 teve a maior mediana nos cinco valores de P. Todos os modos reduziram o tempo ao aumentar P de 2 para 32, mas o benefício incremental diminuiu nos maiores valores de P. A soma global de 33 333 400,000000 é idêntica em todas as execuções, o que verifica a consistência numérica deste caso de teste entre os modos e os números de processos.

6. Discussão

No job inicial sem otimização, uma única execução favoreceu o modo 3 em cerca de 3,3% diante do modo 1. O experimento otimizado e repetido fornece uma base mais ampla para a comparação: o modo 3 liderou pelas medianas em quatro das cinco configurações, com vantagens de 1,3% a 11,2% em relação ao modo 1. A vantagem não cresce monotonicamente com P. Em P=16, o modo 3 variou entre 0,069189 s e 0,111260 s, uma amplitude de 50,8% da sua mediana; em P=32, a amplitude dos três modos ficou entre 28,4% e 40,7% da respectiva mediana. Nessas condições, diferenças pequenas entre medianas exigem cautela. O código testa a conclusão dos dois recebimentos e dos dois envios com MPI_Testall; o custo dessas consultas e a granularidade de 2000 pontos também podem influenciar os tempos. O comportamento do kernel é compatível com acesso intensivo à memória, mas não foram medidas taxas de cache nem largura de banda para estabelecer o gargalo.

A compilação otimizada produziu tempos bem menores que os do job inicial sem flags adicionais (por exemplo, para o modo 1 com P=2: 0,562369 s de mediana, diante de 1,884325 s na execução inicial). Como se trata de submissões distintas, a diferença observada não isola o efeito da otimização: condições do cluster também podem variar. As três repetições consecutivas e a ordem

fixa de modos ajudam a apresentar a dispersão, mas não eliminam efeitos de carga do cluster ou de posição na sequência. O script solicita dois nós e 32 tarefas, sem registrar neste relatório a distribuição efetiva dos ranks por nó.

7. Conclusão

Nas 45 execuções do experimento de escalonamento, as três versões produziram a mesma soma global, 33 333 400,000000. Com compilação -O3 -march=native, o modo 3 apresentou a menor mediana em P=4, 8, 16 e 32; em P=2, a menor mediana foi a do modo bloqueante. O modo com lsend/lrecv seguido de Wait apresentou os maiores tempos medianos em todos os valores de P. Os resultados sustentam uma vantagem da estratégia com cálculo interno durante a comunicação em várias configurações, mas não sustentam a hipótese mais específica de crescimento monotônico dessa vantagem com o número de processos. A dispersão em P=16 e P=32 e a ausência de distribuição de ranks registrada limitam atribuições causais mais precisas. Uma avaliação futura pode ampliar as repetições, alternar a ordem de execução e registrar o mapeamento dos ranks para distinguir melhor o efeito da comunicação das flutuações do ambiente.

Anexo A — Código completo (barra1d.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <mpi.h>
5
6 /*
7  * Difusao de calor 1D com MPI -- comparacao de tres estrategias de
8  * comunicacao para a troca de bordas (halo exchange):
9  *
10  * 1 = MPI_Send / MPI_Recv          (bloqueante)
11  * 2 = MPI_Isend / MPI_Irecv + MPI_Wait (nao-bloqueante, sem overlap)
12  * 3 = MPI_Isend / MPI_Irecv + MPI_Test (nao-bloqueante, COM overlap:
13  *   atualiza os pontos internos em pedacos, testando a comunicacao
14  *   a cada pedaco, e so calcula os pontos de borda quando a
15  *   comunicacao estiver garantidamente completa)
16  */
17
18 #define N_GLOBAL_DEFAULT 1000000
19 #define N_STEPS_DEFAULT 1000
20 #define ALPHA 0.25
21 #define CHUNK_SIZE 2000 /* tamanho do pedaco computado entre cada MPI_Test */
22
23 #define TAG_R 0 /* mensagem "andando para a direita" (enviada ao vizinho direito) */
24 #define TAG_L 1 /* mensagem "andando para a esquerda" (enviada ao vizinho esquerdo) */
25
26 static void atualizar_pontos(const double *u_old, double *u_new,
27                             int start, int end, double c)
28 {
29     for (int i = start; i <= end; i++)
30         u_new[i] = u_old[i] + c * (u_old[i + 1] - 2.0 * u_old[i] + u_old[i - 1]);
31 }
32
33 int main(int argc, char **argv)
34 {
35     int rank, size;
36     MPI_Init(&argc, &argv);
37     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
38     MPI_Comm_size(MPI_COMM_WORLD, &size);
39
40     int mode = (argc > 1) ? atoi(argv[1]) : 3;
41     int N_GLOBAL = (argc > 2) ? atoi(argv[2]) : N_GLOBAL_DEFAULT;
42     int N_STEPS = (argc > 3) ? atoi(argv[3]) : N_STEPS_DEFAULT;
43     if (mode < 1 || mode > 3) mode = 3;
44
45     /* Distribuicao uniforme com resto repartido entre os primeiros processos,
46      * para nao perder pontos quando N_GLOBAL nao e multiplo de size. */
47     int base = N_GLOBAL / size;
48     int resto = N_GLOBAL % size;
49     int n_local = base + (rank < resto ? 1 : 0);
50     int offset = (rank < resto) ? rank * (base + 1)
51                                     : resto * (base + 1) + (rank - resto) * base;
52
53     double *u_old = calloc(n_local + 2, sizeof(double));
54     double *u_new = calloc(n_local + 2, sizeof(double));
55     if (!u_old || !u_new) { MPI_Abort(MPI_COMM_WORLD, 1); }
56
57     /* Condicao inicial: pulso de calor no terco central do dominio global */
58     for (int i = 1; i <= n_local; i++) {
59         int gidx = offset + (i - 1);
60         u_old[i] = (gidx >= N_GLOBAL / 3 && gidx <= 2 * N_GLOBAL / 3) ? 100.0 : 0.0;
61     }
62
63     int left = (rank == 0) ? MPI_PROC_NULL : rank - 1;
64     int right = (rank == size - 1) ? MPI_PROC_NULL : rank + 1;
65
66     if (rank == 0) {
67         const char *nomes[] = {"", "Send/Recv (bloqueante)",
68                                "Isend/Irecv + Wait",
69                                "Isend/Irecv + Test (overlap)"};
70         printf("Modo %d: %s | N=%d P=%d passos=%d\n",
71                mode, nomes[mode], N_GLOBAL, size, N_STEPS);
72     }
```

```

73
74     MPI_Barrier(MPI_COMM_WORLD);
75     double t0 = MPI_Wtime();
76
77     for (int step = 0; step < N_STEPS; step++) {
78
79         if (mode == 1) {
80             /* -----
81              * MODO 1: MPI_Send / MPI_Recv bloqueantes.
82              * A ordem send/recv e escalonada por paridade do rank para evitar
83              * deadlock (todo processo enviando ao mesmo tempo pode travar,
84              * dependendo do protocolo/tamanho da mensagem no MPI usado).
85              * ----- */
86             if (rank % 2 == 0) {
87                 MPI_Send(&u_old[n_local], 1, MPI_DOUBLE, right, TAG_R,
MPI_COMM_WORLD);
88                 MPI_Recv(&u_old[0], 1, MPI_DOUBLE, left, TAG_R, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
89                 MPI_Send(&u_old[1], 1, MPI_DOUBLE, left, TAG_L,
MPI_COMM_WORLD);
90                 MPI_Recv(&u_old[n_local+1], 1, MPI_DOUBLE, right, TAG_L,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
91             } else {
92                 MPI_Recv(&u_old[0], 1, MPI_DOUBLE, left, TAG_R, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
93                 MPI_Send(&u_old[n_local], 1, MPI_DOUBLE, right, TAG_R,
MPI_COMM_WORLD);
94                 MPI_Recv(&u_old[n_local+1], 1, MPI_DOUBLE, right, TAG_L,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
95                 MPI_Send(&u_old[1], 1, MPI_DOUBLE, left, TAG_L,
MPI_COMM_WORLD);
96             }
97
98             atualizar_pontos(u_old, u_new, 1, n_local, ALPHA);
99
100         } else if (mode == 2) {
101             /* -----
102              * MODO 2: MPI_Isend / MPI_Irecv seguido de MPI_Wait imediato.
103              * Nao ha sobreposicao: a CPU fica ociosa esperando a comunicacao
104              * terminar antes de comecar a computar qualquer ponto.
105              * ----- */
106             MPI_Request reqs[4];
107             MPI_Irecv(&u_old[0], 1, MPI_DOUBLE, left, TAG_R, MPI_COMM_WORLD,
&reqs[0]);
108             MPI_Irecv(&u_old[n_local+1], 1, MPI_DOUBLE, right, TAG_L, MPI_COMM_WORLD,
&reqs[1]);
109             MPI_Isend(&u_old[n_local], 1, MPI_DOUBLE, right, TAG_R, MPI_COMM_WORLD,
&reqs[2]);
110             MPI_Isend(&u_old[1], 1, MPI_DOUBLE, left, TAG_L, MPI_COMM_WORLD,
&reqs[3]);
111
112             MPI_Wait(&reqs[0], MPI_STATUS_IGNORE);
113             MPI_Wait(&reqs[1], MPI_STATUS_IGNORE);
114             MPI_Wait(&reqs[2], MPI_STATUS_IGNORE);
115             MPI_Wait(&reqs[3], MPI_STATUS_IGNORE);
116
117             atualizar_pontos(u_old, u_new, 1, n_local, ALPHA);
118
119         } else {
120             /* -----
121              * MODO 3: MPI_Isend / MPI_Irecv + MPI_Test em loop.
122              * Enquanto a comunicacao das bordas esta em transito, calculamos
123              * os pontos INTERNOS (que nao dependem das ghost cells) em
124              * pedacos de CHUNK_SIZE, testando a cada pedaco se a comunicacao
125              * ja terminou. So os pontos de BORDA, que dependem das ghost
126              * cells recebidas, esperam a confirmacao (MPI_Test/Waitall) de
127              * que a comunicacao de fato concluiu.
128              * ----- */
129             MPI_Request reqs[4];
130             MPI_Irecv(&u_old[0], 1, MPI_DOUBLE, left, TAG_R, MPI_COMM_WORLD,
&reqs[0]);
131             MPI_Irecv(&u_old[n_local+1], 1, MPI_DOUBLE, right, TAG_L, MPI_COMM_WORLD,
&reqs[1]);
132             MPI_Isend(&u_old[n_local], 1, MPI_DOUBLE, right, TAG_R, MPI_COMM_WORLD,
&reqs[2]);
133             MPI_Isend(&u_old[1], 1, MPI_DOUBLE, left, TAG_L, MPI_COMM_WORLD,
&reqs[3]);

```

```

134
135         int interior_end = n_local - 1;    /* ultimo ponto interno (nao depende de
ghost) */
136         int next = 2;                      /* primeiro ponto interno */
137         int comm_done = 0;
138
139         while (next <= interior_end) {
140             int chunk_end = next + CHUNK_SIZE - 1;
141             if (chunk_end > interior_end) chunk_end = interior_end;
142
143             atualizar_pontos(u_old, u_new, next, chunk_end, ALPHA);
144             next = chunk_end + 1;
145
146             MPI_Testall(4, reqs, &comm_done, MPI_STATUSES_IGNORE);
147         }
148
149         /* Garante que a comunicacao terminou antes de tocar nas ghost cells */
150         if (!comm_done)
151             MPI_Waitall(4, reqs, MPI_STATUSES_IGNORE);
152
153         /* Pontos de borda, que dependem das ghost cells recém-recebidas */
154         atualizar_pontos(u_old, u_new, 1, 1, ALPHA);
155         atualizar_pontos(u_old, u_new, n_local, n_local, ALPHA);
156     }
157
158     double *tmp = u_old; u_old = u_new; u_new = tmp;
159 }
160
161 MPI_Barrier(MPI_COMM_WORLD);
162 double t1 = MPI_Wtime();
163 double t_local = t1 - t0, t_max;
164 MPI_Reduce(&t_local, &t_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
165
166 double soma_local = 0.0;
167 for (int i = 1; i <= n_local; i++) soma_local += u_old[i];
168 double soma_global;
169 MPI_Reduce(&soma_local, &soma_global, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
170
171 if (rank == 0)
172     printf(" -> tempo=%.6f s | soma_global=%.6f\n", t_max, soma_global);
173
174 free(u_old);
175 free(u_new);
176 MPI_Finalize();
177 return 0;
178 }
179

```

Anexo B — Script SLURM (job 2118946)

```
1 #!/bin/bash
2 #SBATCH --job-name=tarefa_15
3 #SBATCH --time=0-0:10
4 #SBATCH --partition=amd-512
5 #SBATCH --nodes=2
6 #SBATCH --ntasks=2
7 #SBATCH --cpus-per-task=1
8 #SBATCH --output=%x_%j.out
9 #SBATCH --error=%x_%j.err
10
11 mpicc -o barra barraId.c
12 mpirun -np 2 ./barra 1 1000000 1000 # Send/Recv
13 mpirun -np 2 ./barra 2 1000000 1000 # Isend/Irecv + Wait
14 mpirun -np 2 ./barra 3 1000000 1000 # Isend/Irecv + Test
15
```

Anexo C — Script SLURM de escalonamento (job 2121424, script.sh)

```
1 #!/bin/bash
2 #SBATCH --job-name=tarefa_15_scaling
3 #SBATCH --time=0-0:30
4 #SBATCH --partition=amd-512
5 #SBATCH --nodes=2
6 #SBATCH --ntasks=32
7 #SBATCH --cpus-per-task=1
8 #SBATCH --output=%x_%j.out
9 #SBATCH --error=%x_%j.err
10
11 # Compilar com otimizacao -- sem -O3 o tempo de computacao fica maior,
12 # distorcendo a proporcao comunicacao/computacao que estamos comparando.
13 mpicc -O3 -march=native -o barra barraId.c -lm
14
15 N=1000000
16 STEPS=1000
17 REPS=3 # repeticoes por combinacao, para filtrar ruido do cluster
compartilhado
18
19 for P in 2 4 8 16 32; do
20     for MODO in 1 2 3; do
21         for REP in $(seq 1 $REPS); do
22             echo "--- P=$P MODO=$MODO REP=$REP ---"
23             mpirun -np $P ./barra $MODO $N $STEPS
24         done
25     done
26 done
```